

Vision, Market Strategy & Cursor-for-Data Build Plan

Product: **Que** (SchemaGraph) · Codebase: `adc/schemagraph`

Document version: **1.0** · Date: 9 Aug 2026

Audience: founders, investors, product, sales engineers · Companion: `Que-Problems-And-User-Guide.pdf`, `Que-Built-Roadmap-Compete.pdf`

Long-term motive: Become the *AI copilot + workflow OS* for data engineers and analytics — so connecting sources, discovering joins, building models, validating, shipping to warehouse/BI, and keeping things from drifting becomes as fluid as coding in **Cursor** — for the *whole* DE/DA team, not only senior DEs.

Today's honest product: Schema + HITL join truth + attested delivery into *their* systems — not Que-as-warehouse, not “replace Databricks,” not full catalog yet.

Rule: Sell the wedge now. Roadmap the Cursor-for-data category. Never sell Phase-4 language with Phase-0 software.

Table of contents

1. Aim, motive & north star
2. What Que has today (features)
3. How we are different in the market
4. Competitors & positioning
5. How to sell Que (story, buyers, POC)
6. Competing with Databricks (realistic path)
7. Cursor for data teams — what that means
8. Phased build plan (0 → 5)
9. What more to build (detailed backlog)
10. Success metrics & claim gates
11. Risks & what never to claim early

1. Aim, motive & north star

1.1 Aim (what we exist to do)

Remove the hardest, most tribal, most error-prone work in modern data teams: **understanding how systems connect, approving those connections, and shipping trusted work into the warehouse / BI / orchestrator** — without forcing the company to give a vendor custody of their data lake.

1.2 Motive (why we build Que)

- **Senior DEs are bottlenecks** — keys, joins, and “how we stitch finance ↔ CRM” live in their heads.
- **Catalogs inventory; pipelines still break** — knowing a table exists ≠ shipping a correct join.
- **AI is blocked by security** — CISOs will not “send the lake” to a chatbot.
- **Platforms are silos** — Databricks/Snowflake are excellent *inside* one lakehouse; real companies are multi-source.
- **Teams deserve Cursor-like leverage** — analysts and junior DEs should move as fast as senior people, with guardrails.

1.3 North star (Cursor for data teams)

Become the **AI copilot + workflow OS** for data engineers and analytics — so connecting sources, discovering joins, building models, validating, shipping to warehouse/BI, and keeping things from drifting becomes as fluid as coding in Cursor — for the whole DE/DA team, not only senior DEs.

Cursor (software engineers)	Que (data engineers + analysts)
Edit code with AI assist	Edit stitch / models with AI assist
Human reviews before merge	Human Promotes joins / freezes contracts
Git is source of truth	Warehouse + dbt + Airflow stay SoR
IDE is not the production runtime	Que is not the data lake
Whole team ships faster	Whole DE/DA team ships trusted data faster

Analogy discipline: Cursor accelerates writing code *with review*. Que must accelerate data work *with Promote / freeze* — a wrong join is worse than a wrong autocomplete.

2. What Que has today (features)

Everything below is implemented and demoable in the current stack (Vite SPA + Express API + Postgres).

Area	What users can do	Status
Access & tenancy	Sign up / sign in; optional OIDC SSO; create/switch workspaces; invite members; roles viewer → member → admin → owner; onboarding roadmap	BUILT
Sources	PostgreSQL, MongoDB, Excel/CSV, Snowflake, Databricks, BigQuery, Salesforce; encrypted credentials; sync schema (+ capped samples); connection health; source sync schedules (hourly/daily)	BUILT
Schema canvas	Interactive graph; filters (relationship type, confidence, source type); search; Promote/Reject on edges; stitch session; export PDF/PNG/JSON	BUILT
Join Review	Inbox Pending/Accepted/Rejected; re-run inference; mapping-assist Suggest mappings + Accept/Reject/Dismiss	BUILT
AI Chat	Schema-only skills (/list, /describe, /joins, /sql, /job, /diff, /privacy...); RAG reindex; draft job → Save to Jobs	BUILT
Jobs	Notebook cells; dry-run (~10 sample rows); live validate (~20 read-only); contract freeze; drift ack; attested export JSON/SQL/dbt; optional GitHub PR; signed download URLs; materialize VIEW/table in customer warehouse; job schedules + retries; private-runner execution target	BUILT
Lineage lite	Metadata path Sources → joins → job → export/materialize	BUILT
Trust & ops	Audit log; drift alerts; attestation verify (/verify); policy gates (block export on unreviewed joins / high drift); orchestrator webhook (Airflow/Dagster); soft seat billing; usage bars	BUILT

Product truth: Que stores schema + optional capped/scrubbed samples — not full tables. Promote is the join gate. Warehouse remains system of record for result rows.

2.1 Problems we solve now (buyer language)

1. **“We can’t safely stitch across systems”** — schema sync + suggest + human Promote.
2. **“Catalogs inventory; nobody ships a stitch job”** — freeze → notebook → attested export / materialize.
3. **“Security won’t let AI touch our lake”** — schema-first + capped/scrubbed samples.
4. **“Who changed what / can we prove this export?”** — roles, audit, drift, HMAC attestation.
5. **“Schema drifts and joins break silently”** — drift surfaces + export blocks.
6. **“Paste into Airflow? Then what?”** — signed artifacts, webhooks, private runner, customer CTAS/VIEW.

3. How we are different in the market

Dimension	Typical market	Que difference
Core loop	Search / catalog / chat SQL	Suggest → Promote → Freeze → Attested ship
Data custody	Often wants broad row access or lives in one platform	Schema-first; capped samples; no Que lake
Cross-source	Strong inside one lakehouse	Multi-source by design (ops DB + files + CRM + warehouse)
AI posture	Auto-answer / auto-SQL	HITL mandatory where trust matters (joins/contracts)
Delivery	Stops at diagram or Slack dump	Artifact / PR / materialize / orchestrator handoff
Who wins day-1	Platforms win if already all-in on them	Que wins when reality is messy and multi-system

Wedge sentence: We don't replace your warehouse. We make cross-source stitch *correct, reviewable, and shippable* — then deliver into systems you already trust.

4. Competitors & positioning

Layer	Players	vs Que (today)
Catalog / governance	Atlan, Alation, Collibra, DataHub	They inventory & steward. Que ships approved join contracts . Do not claim catalog replacement this quarter.
Lakehouse / platform	Databricks , Snowflake	Where data lives & runs. Que sits in front / beside — stitch + handoff into them.
ELT / transform	Fivetran, Airbyte, dbt	They move/transform. Que decides which joins are trusted before/alongside transforms.
Orchestration	Airflow, Dagster, Prefect	They run DAGs. Que triggers / packs work for them; does not host Airflow.
AI for data	Platform AI/BI, text-to-SQL tools	Often one-platform or chat-only. Que = cross-source + HITL + attestation .
Future "Cursor for data"	Notebook copilots, agent startups	Overlap with endgame. Win by shipping the trusted loop first.

4.1 Competing now vs later

Competing now (wedge)	Not competing yet
Manual join discovery across systems Tribal knowledge of keys in spreadsheets Hand-written dbt joins with no attestation Catalogs that stop at "search" without a ship path	Atlan/Collibra as catalog of record Unity Catalog / Horizon as control plane Full BI semantic layer platforms Big-4-audited unattended multi-tenant SaaS

5. How to sell Que

5.1 Pain narrative (open with this)

"Our DE and analytics time dies in three places: finding keys across systems, wrong joins reaching production, and security blocking AI from the lake. Catalogs tell us what exists. Platforms store the data. Nobody owns *trusted stitch* → *ship*."

5.2 Offer (what you sell this quarter)

- Paid design-partner / technical POC with **fixed SOW** (2–3 of *their* sources)
- Success = **N promoted joins** + **1 attested export** (or materialize) + precision/recall on *their* golden join pairs
- **Not** success = "catalog coverage %" or "replace Atlan"

5.3 Buyer map

Persona	What they care about	What to show
Head of Data / DE lead	Wrong joins, hours wasted, ship path	Promote → freeze → export/PR/materialize
CISO / Security	No lake custody; auditability	Schema-only, scrub, roles, attestation verify
Analytics lead	Trusted metrics paths without waiting months	Chat draft + reviewed job → warehouse object
Platform owner (DBX/SF)	Don't rip-and-replace	"We deliver into your warehouse"

5.4 Ideal first customers

- Mid-market / growth companies with **multi-source mess** (ops DB + warehouse + files + CRM)
- Teams already on Databricks or Snowflake but still stitching from Postgres/Mongo/Excel/Salesforce
- **Avoid first:** "100% Unity Catalog, everything already inside one platform, wants full Colibra replacement"

6. Competing with Databricks (realistic path)

You do **not** out-compute Databricks. You out-workflow the multi-source stitch + team productivity layer.

Don't	Do
Claim "replace Databricks / Unity Catalog"	Be the best connector + join HITL + attested ship into Databricks
Host their lake	Materialize / dbt / webhook into Databricks
Auto-accept joins	Prove human gate + evidence beats silent AI mistakes
Vague "AI platform"	Ship measurable DE hours saved on stitch → validate → deploy

6.1 Databricks-specific build priorities

- Deepen Unity / metastore schema introspect quality
- Expand query-history join mining (already a wedge signal)
- First-class materialize + dbt ship into Databricks SQL warehouses
- Orchestrator webhook examples for Databricks Workflows / Airflow-on-DBX
- Case study: "Postgres + Salesforce + Databricks → promoted joins → attested model in DBX"

7. Cursor for data teams — product vision in detail

7.1 The full loop we eventually own

1. Connect sources	(schema sync, health, schedules)
2. Discover relationships	(AI/heuristic + evidence)
3. Human approve	(Promote / Reject / mapping assist)
4. Build models / jobs	(notebook, SQL, dbt draft – AI assisted)
5. Validate	(dry-run samples, live validate, tests)
6. Ship	(export, PR, materialize, webhook)
7. Observe drift	(schema/join/contract drift alerts)
8. Fix with assist	(suggested remaps, PRs, re-freeze)
↑ _____	team memory of accepted joins / mappings _____

Today Que strongly covers **1–3 and parts of 4–7**. The Cursor-level product completes **4–8 as an agentic, team-wide OS** — still with HITL checkpoints, still without hosting the lake.

7.2 Who the product is for (whole team)

Role	Cursor-like experience they should get
Senior DE	Agent proposes stitch plans; they Promote/freeze; less tribal firefighting
Junior DE	Guided join review + notebook drafts with guardrails; learn by approving evidence
Analytics engineer	dbt model drafts from approved contracts; CI shows attestation
Analyst	“Ask → trusted path → BI object” without writing brittle cross-DB SQL alone
Data lead / steward	Audit, drift, who approved what; policy gates

7.3 Non-negotiable DNA (even at Cursor scale)

- Que is **not** the system of record for production rows
- Joins that become contracts require **human Promote** (or explicit policy exception)
- AI may draft; humans (or policy) gate ship
- Delivery lands in **customer** warehouse / git / orchestrator
- Schema-first defaults; samples capped/scrubbed

8. Phased build plan (how to get there)

Each phase has an **honest claim unlock**. Do not market the next phase's claim until exit criteria pass.

PHASE 0 Harden the wedge (now → ~1–2 quarters)

Goal: Paid POCs close reliably. Join trust is demo-proof. Ship path is boring.

Claim unlock: "Controlled paid POC for cross-source join HITL → attested export/materialize."

Build

- Join **evidence UX** on every suggested edge (signals, samples overlap, optional SQL)
- Customer **golden-set eval** → printable POC report (precision/recall)
- One-click POC pack (fixtures, migrate, verify attestation)
- Hide billing/upgrade stubs outside DEV; polish Settings credibility
- Audit log export, session revoke, basic rate limits
- Snowflake query-history join mining (parity with Databricks assist)
- Deeper dbt manifest round-trip; GitHub check with attestation fingerprint + approved join IDs
- Live connector quality: BigQuery, Salesforce, Databricks, Snowflake hardening

Exit criteria

- ≥3 design partners run golden-set POC with documented join quality
- SE can demo Promote → freeze → attested artifact without apology
- Materialize or dbt PR works on at least one customer warehouse end-to-end

PHASE 1 Copilot that drafts the stitch (Cursor Step 1)

Goal: Multi-step agent proposes work; humans approve at checkpoints — like Cursor proposing an edit.

Claim unlock: "AI proposes cross-source stitch plans; humans Promote; Que drafts the job."

Build

- **Stitch Agent:** "Connect domain X (orders ↔ customers ↔ payments)" → plan → suggest joins → wait for Promote → draft notebook/dbt
- Checkpoint UI: Accept plan / edit plan / reject step (never silent auto-ship)
- Org **join memory**: accepted joins & mappings reusable across jobs/workspaces
- Better Chat → Jobs quality (tests, assumptions, cited tables)
- Side-by-side evidence panel while agent runs
- Analyst-safe mode: limited skills, no credential access, read-oriented

Exit criteria

- Agent completes a 2-source stitch to drafted job with ≥1 human checkpoint in <30 minutes in POC
- Junior DE can Promote with evidence without senior sitting beside them

PHASE 2 Workflow OS for the whole DE/DA team

Goal: Que becomes daily home for stitch → model → ship → drift — not a one-time POC tool.

Claim unlock: “Team workflow OS for trusted stitch-to-ship across sources.”

Build

- Shared workspaces by **domain / data product** (orders, finance, growth)
- Review queues: joins, mappings, contract freezes, drift fixes (Slack/Teams notify)
- Metrics & semantic stubs: approved metrics linked to frozen contracts
- BI handoff: create/update dbt models or warehouse views that analysts query in Looker/PowerBI/Tableau
- Job templates library (“SCD2 dim”, “fact orders stitch”, “CRM enrich”)
- Permissions: who can Promote vs who can only propose
- Usage analytics: hours saved, joins promoted, exports shipped

Exit criteria

- Paying teams use Que weekly (not only during POC)
- Both DE and analytics roles have distinct happy paths in the same workspace

PHASE 3 Full Cursor-like loop (automate the bottleneck end-to-end)

Goal: From NL intent to warehouse/BI object with continuous drift repair — still HITL at trust gates.

Claim unlock: "Cursor for data teams: intent → reviewed stitch → ship → auto-detect drift → assisted fix."

Build

- Natural language: "Build trusted customer 360 from SF + Postgres + Databricks" → multi-agent plan
- Validation suite generator: uniqueness, referential checks, row-count sanity (run in customer warehouse)
- Drift agent: schema change → impact on contracts → suggested remap PR → re-freeze
- Private runner / BYOC agent maturity (VPC) beyond callback MVP
- Eval harnesses for hallucination; contractual schema-only guarantees
- Grounded answers with citations to lineage edges + owners
- Optional auto-Promote only for *low-risk* joins under explicit policy (default still HITL)

Exit criteria

- Design partners report measurable reduction in DE cycle time for new domain stitches
- Drift-to-fix PR path used in production by ≥ 1 customer

PHASE 4 Optional expansion — catalog / governance (only after wedge revenue)

Goal: Compete with Atlan/Collibra *if* strategy expands — not required for Cursor-for-data.

Claim unlock: "Catalog + governance platform" — only when built.

Build (if pursuing)

- Connector breadth at scale; dashboards/metrics/pipelines as first-class assets
- Business glossary; stewardship workflows; certify/expire
- Automated multi-hop column lineage across SQL + dbt + BI
- Policy packs, Jira/ServiceNow, steward UX mode

Do not let Phase 4 steal focus from Phases 0–3. Catalog parity is expensive and slower to monetize than the Cursor wedge.

PHASE 5 Enterprise control plane (Big-4 bar)

Goal: Unattended multi-tenant SaaS sellable into heavily audited enterprises.

Build

- SCIM 2.0; enforced SSO; CMK; fine-grained RBAC/ABAC
- SOC 2 Type II evidence pack; pen-test; contractual SLAs
- Multi-region / tenant isolation story; SIEM export

9. What more to build (detailed backlog)

Organized by layer. Priority = Phase 0 first, then Cursor path (1–3), then optional category (4–5).

9.1 Trust & join quality **P0**

Item	Why
Evidence panel on every edge (name match, type match, sample overlap, FK declaration, query-history hit)	Trust hinge
Golden-set upload + score report PDF	POC proof
False-positive review workflow (“why rejected” memory)	Model improves with team
Composite / multi-column join support UX	Real-world keys
Conflict UI when two promoted joins disagree	Avoid silent wrong truth

9.2 Agent / Cursor core **P1**

Item	Why
Multi-step Stitch Agent with checkpoints	Cursor-like flow
Plan editor (reorder/remove steps before run)	Human control
Tool use: sync, infer, promote request, draft SQL, open PR (permissions gated)	Real automation
Session transcript + replay for audits	Enterprise trust
“Ask → job” for analysts with restricted tools	Whole-team product

9.3 Ship & runtime **P1–P2**

Item	Why
dbt project round-trip (read existing models, write only delta)	Lives in their repo
CI GitHub Action: attestation + approved join IDs	Non-repudiation
Warehouse test runner (customer-side checks)	Validate for real
OpenLineage / Airflow richer payloads	Orchestrator depth
BYOC private agent (beyond callback MVP)	VPC customers
Idempotent materialize + rollback notes	Ops safety

9.4 Team OS **P2**

Item	Why
Domain spaces + ownership	Scale beyond one canvas
Slack/Teams: “join needs Promote”, drift digests	Meet teams where they work
Template gallery + org private templates	Speed for juniors
Propose vs Promote permissions	Analyst vs DE roles
Saved “data products” with versioned contracts	Product thinking

9.5 Analytics experience **P2–P3**

Item	Why
Metric definitions bound to frozen contracts	Trusted numbers
NL → reviewed SQL against approved graph only	Safer than open text-to-SQL
Explain “why this join” in plain language	Analyst literacy
BI tool deep links (Looker explore / Power BI dataset)	Close the loop to consumption

9.6 Platform & enterprise P3-P5

Item	Why
More connectors: Redshift, Synapse, Glue, Kafka, S3 tables...	Coverage
SCIM, CMK, SOC2, SIEM export	Enterprise buy
Optional glossary / stewardship (only if category expand)	Catalog competition
Multi-region tenancy evidence	Big-4 bar

9.7 Explicitly do *not* build early

- Que-hosted data lake / scratch warehouse as SoR
- Auto-accept all joins by default
- AI “clean the whole lake” row mutation inside Que
- Full Airflow/Databricks clone
- Full Collibra/Atlan replacement before wedge revenue

10. Success metrics & claim gates

Phase	North-star metric	You may claim when...
0 Wedge	POC win rate; join precision/recall on golden set; time-to-first attested export	Paid POC SOW with fixed sources succeeds repeatedly
1 Copilot	% of jobs that started from agent draft; human checkpoint rate; junior DE success	Agent drafts used in real POCs without senior babysitting
2 Team OS	Weekly active DE+analyst seats; joins promoted / week; exports shipped / week	Customers renew because Que is in the weekly workflow
3 Cursor loop	Cycle time for new domain stitch; drift MTTR; % fixes via assisted PR	Customers measure hours saved on stitch→ship→repair
4 Catalog (opt)	Asset coverage; steward adoption	Only after intentional strategy expand + feature parity
5 Enterprise	SOC2; SCIM customers; uptime SLA	Evidence pack exists — not slideware

11. Risks & what never to claim early

Risk	Mitigation
Overclaim vs Atlan/Databricks	Wedge language only; “alongside” not “replace”
AI wrong joins in production	Mandatory Promote; block export on unreviewed; evidence UX
CISO kills deal	Schema-only defaults; scrub; no lake custody; attestation
Becoming a catalog science project	Phases 0–3 before Phase 4
“Cursor” hype without checkpoints	Agent always pauses at trust gates
Trying to host compute	Materialize + webhook + BYOC; never Que-as-warehouse

Final line for stakeholders:

Que’s *motive* is to become the Cursor-like OS for data engineering and analytics. Que’s *current product* is the sharp first chapter: cross-source join truth + safe AI + attested delivery into systems customers already own — especially beside Databricks and Snowflake, not instead of them.

Build order: Harden wedge → Stitch agent with HITL → Team workflow OS → Full intent→ship→drift loop → (optional) catalog/enterprise control plane.